

Lecture 7: Error Detection and Correction

WHY ERROR DETECTION AND CORRECTION

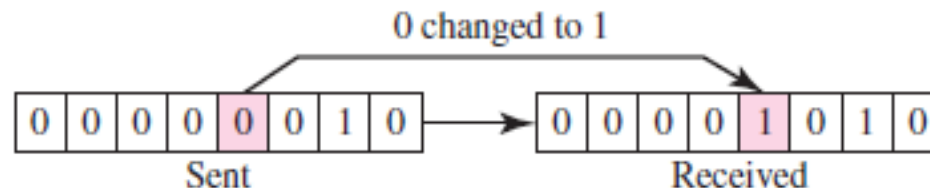
- Because of attenuation, distortion, noise and interferences, errors during transmission are inevitable, leading to corruption of transmitted bits.
- The longer the frame size, the higher the probability of single bit error and the lower the probability of receiving a frame without error.
- This clearly emphasises the need for error detection and error correction.

TYPES OF ERRORS

- Error occurs when a bit is altered between transmission and reception, i.e. a binary 1 is transmitted and a binary 0 is received.
- Two types of errors

Single bit error

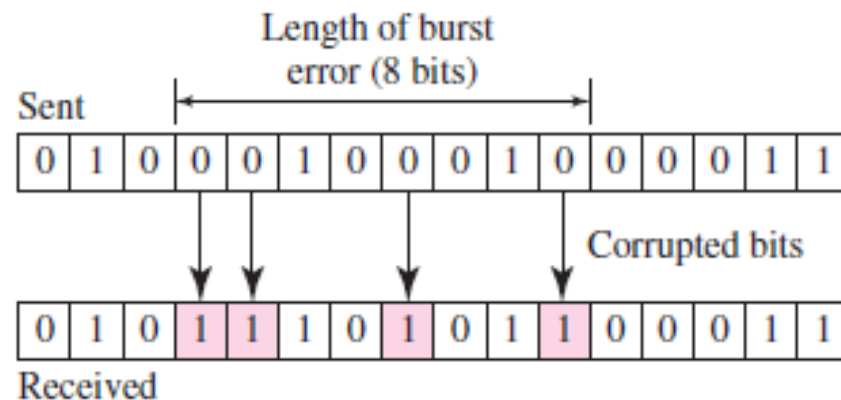
- Only one bit gets corrupted, means that only 1 bit of a given data unit (such as a byte, character, or packet) is changed from 1 to 0 or from 0 to 1.
- Common used in parallel transmission.



Burst error:

- More than one bit get corrupted. means that 2 or more bits in the data unit have changed from 1 to 0 or from 0 to 1.
- More common in serial transmission of data. Occurs when the duration of noise is longer than the duration of one bit.

Example: *Burst error of length 8*

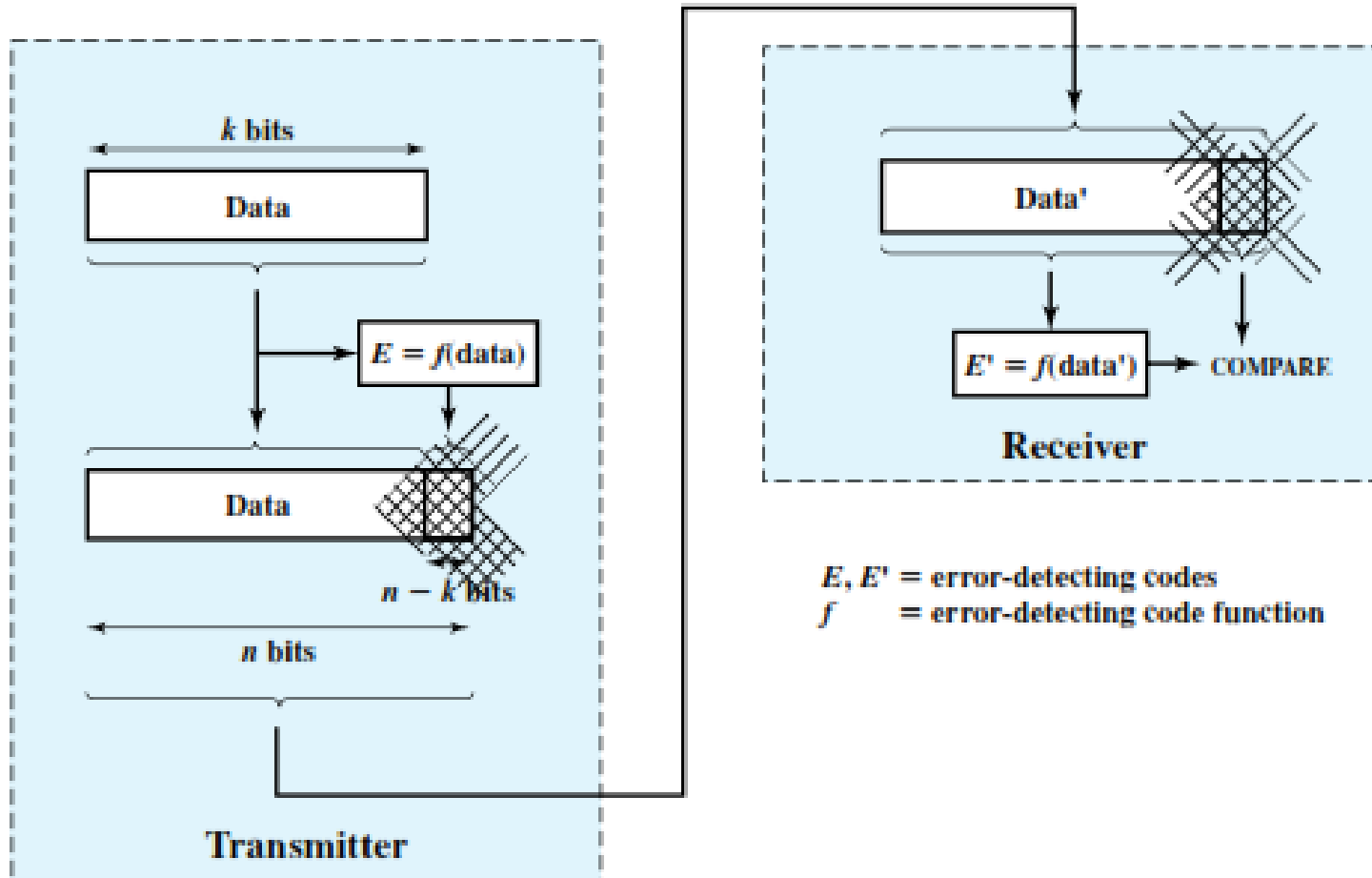


ERROR DETECTION TECHNIQUES

- Regardless of the design of the transmission system, there will be errors, resulting in the change of one or more bits in a transmitted frame.
- The probability that a frame arrives with no bit errors decreases when the probability of a single bit error increases.
- The probability that a frame arrives with no bit errors decreases with increasing frame length.

ERROR DETECTION TECHNIQUES Ctd...

Error Detection Process

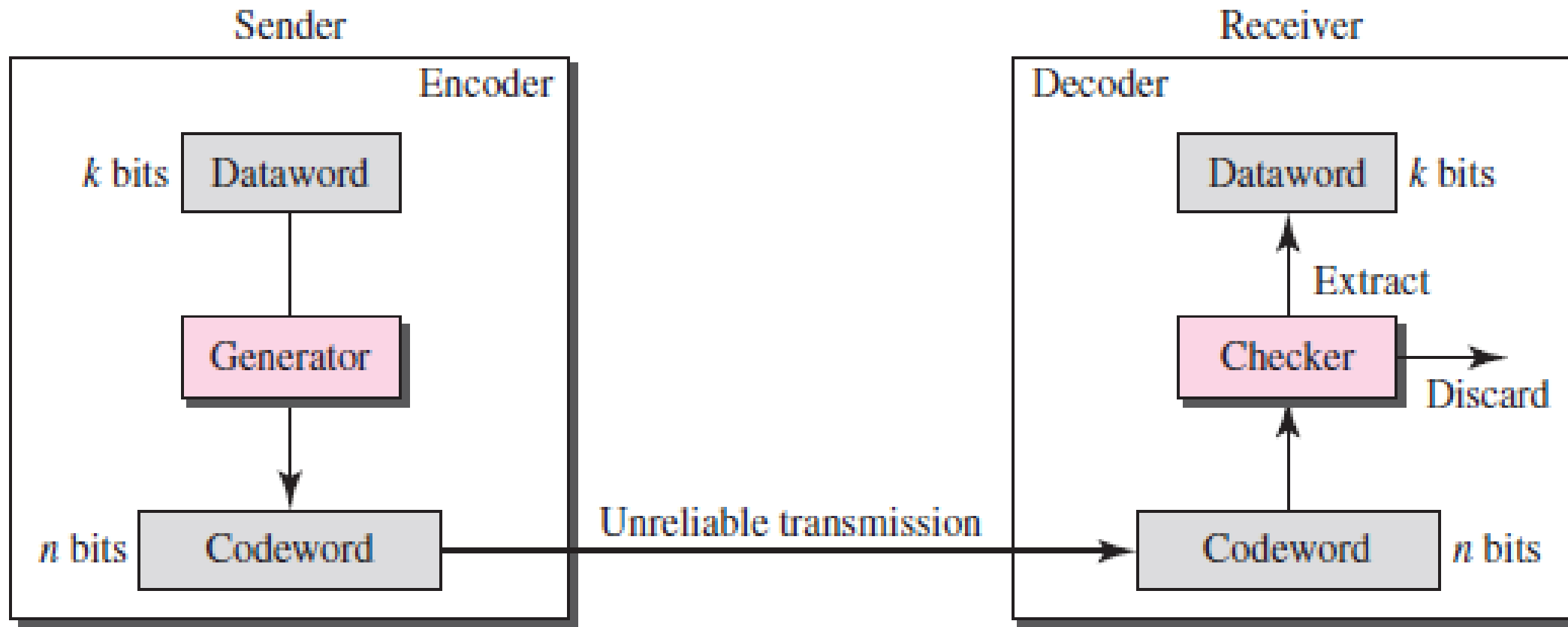


- An error detecting code is added by a transmitter
- The error detecting code is checked and re-calculated by a receiver and compare it with the value of the incoming error detection code.
- A detected error occurs if and only if there is a mismatch.

- The central concept in detecting or correcting errors is **redundancy**.
- To be able to detect or correct errors, we need to send some extra bits with our data.
- These redundant bits are added by the sender and removed by the receiver.
- Their presence allows the receiver to detect or correct corrupted bits

Note

To detect or correct errors, we need to send extra (redundant) bits with data

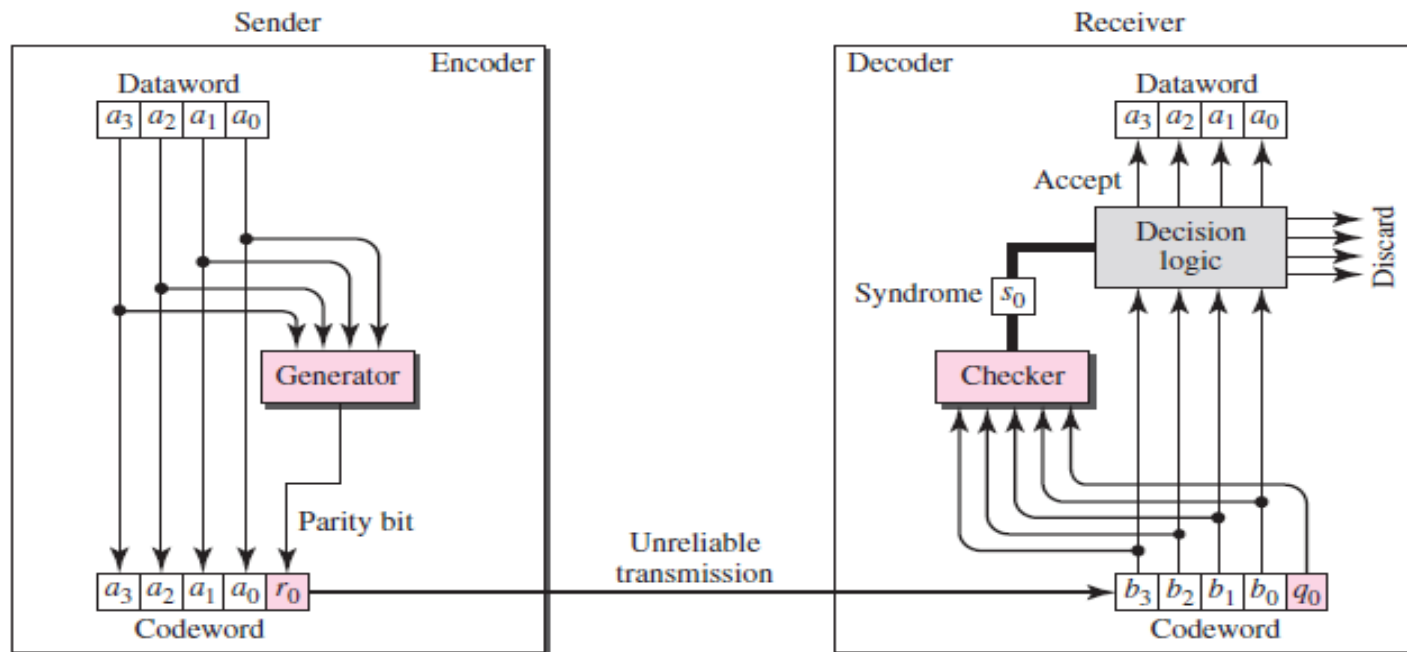


TYPES OF ERROR DETECTION TECHNIQUES

- **Popular Techniques:**
 - Simple Parity Check
 - Two dimensional Parity Check
 - Checksum
 - Cyclic Redundancy Check (CRC)

SIMPLE PARITY CHECK

- The simplest and the most popular error detection scheme
- Appends a parity bit to the end of the data either 0 or 1.
- The value of the bit is selected so that the character has an even number of 1s (**even parity**) or an odd number of 1s (**odd parity**)



$$r_0 = a_3 + a_2 + a_1 + a_0 \quad (\text{modulo-2})$$

$$s_0 = b_3 + b_2 + b_1 + b_0 + q_0 \quad (\text{modulo-2})$$

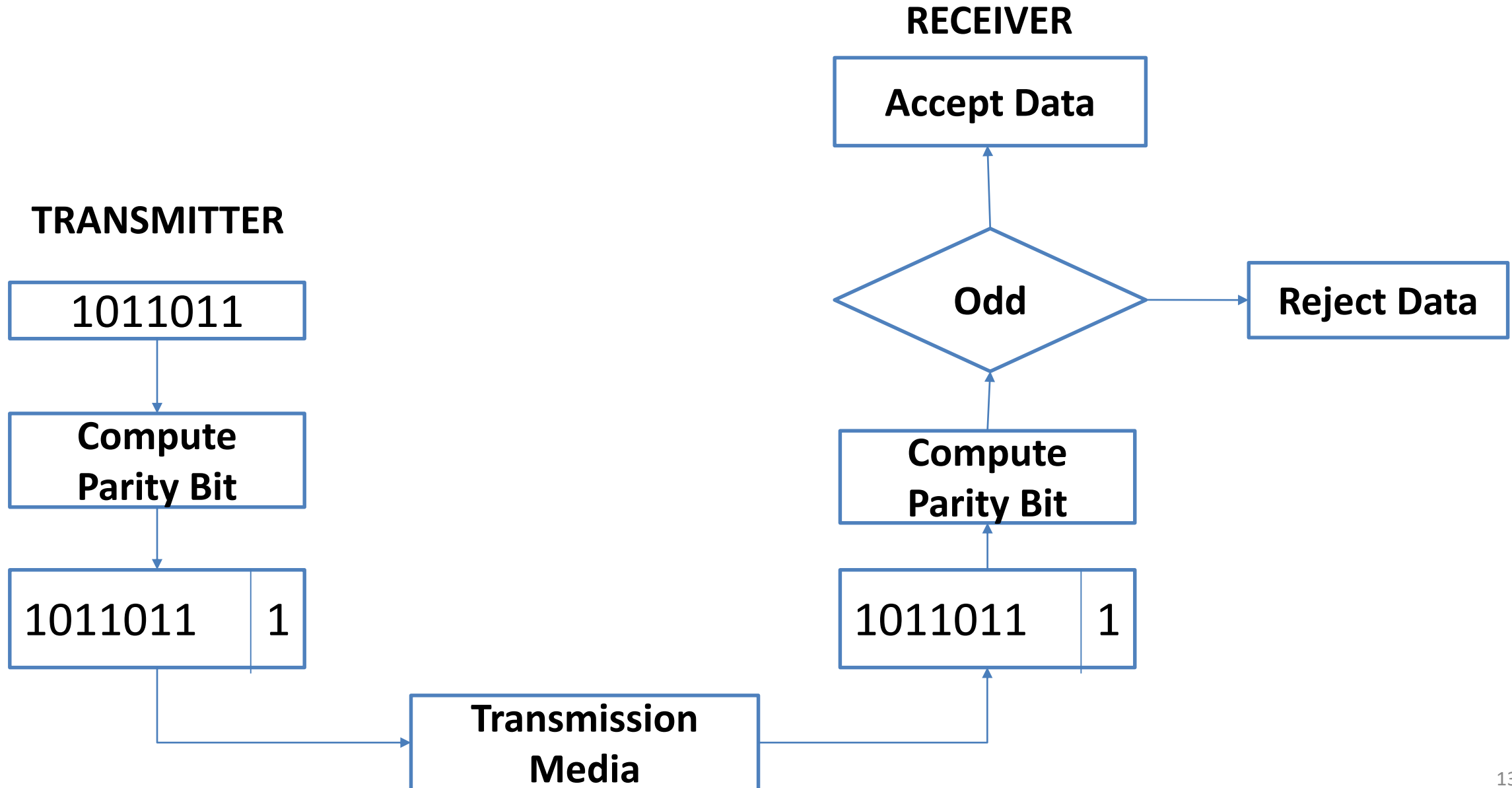
Datawords and Codewords

<i>Datawords</i>	<i>Codewords</i>	<i>Datawords</i>	<i>Codewords</i>
0000	00000	1000	10001
0001	00011	1001	10010
0010	00101	1010	10100
0011	00110	1011	10111
0100	01001	1100	11000
0101	01010	1101	11011
0110	01100	1110	11101
0111	01111	1111	11110

Example

- If the transmitter is transmitting a character **G (1011)** and using odd parity, it will append a 1 and transmit **10111**.
- The receiver examines the received character and, if the total number of 1s is odd, assumes that no error has occurred. If one bit (or any odd number of bits) is erroneously inverted during transmission (for example, **10111**), then the receiver will detect an error

SIMPLE PARITY CHECK Ctd...

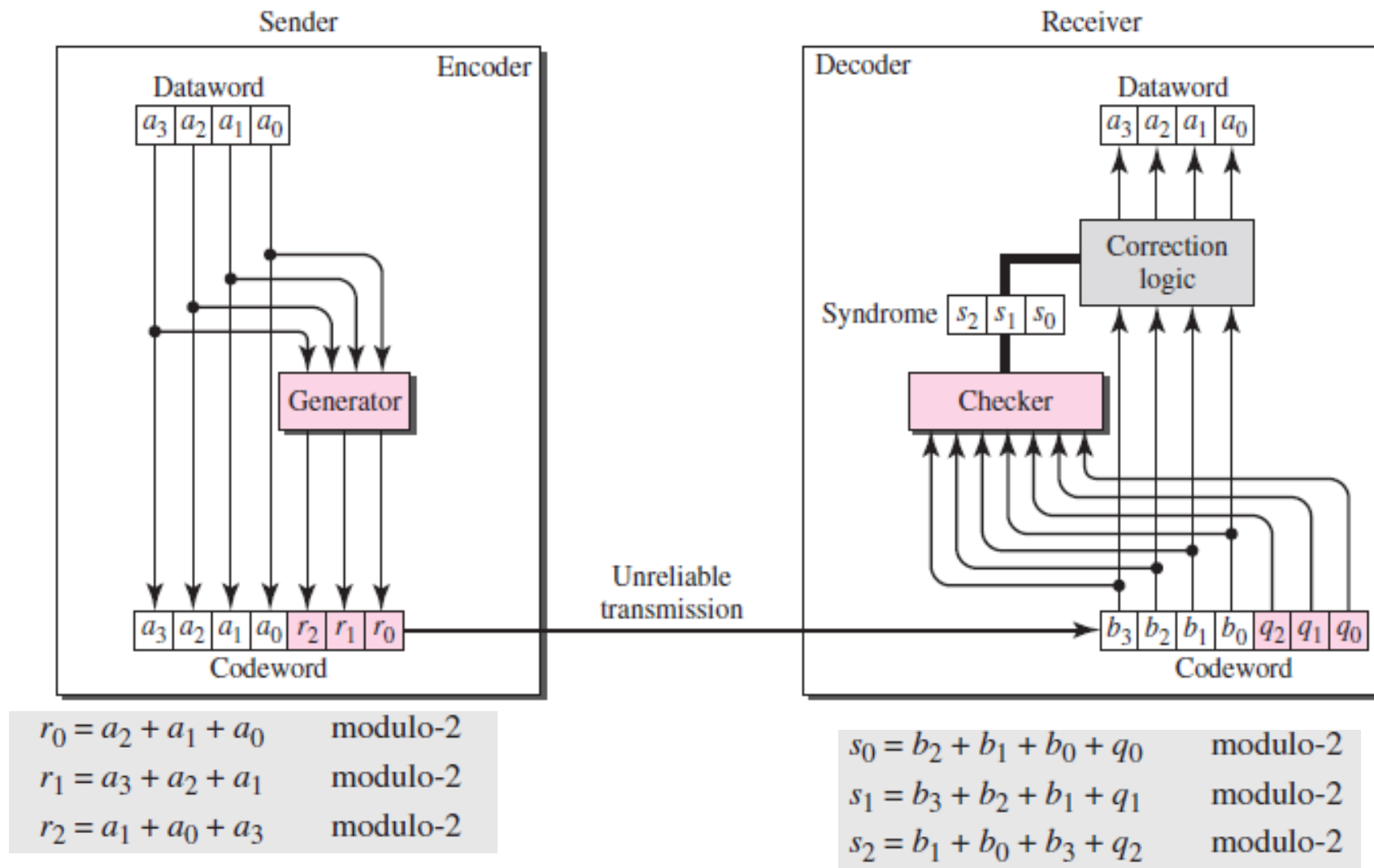


SIMPLE PARITY CHECK Ctd...

- **Performance of Simple Parity Check**
 - It can detect all single bit errors
 - The technique is not fool proof against burst errors that inverts more than one bits.
 - If even number of bits are inverted due to error, the error is not detected.

TWO DIMENSIONAL PARITY CHECK

- Two dimensional parity check organizes the block of bits in the form of a table
- Parity check bits are calculated for each row, which is equivalent to a simple parity check bit
- Parity check bits are also calculated for all columns
- Both are sent along with the data
- At the receiving end these are compared with the parity bits calculated on the received data.



Modulo-2 Arithmetic

Of particular interest is modulo-2 arithmetic. In this arithmetic, the modulus N is 2. We can use only 0 and 1. Operations in this arithmetic are very simple. The following shows how we can add or subtract 2 bits.

Adding:	$0+0=0$	$0+1=1$	$1+0=1$	$1+1=0$
Subtracting:	$0-0=0$	$0-1=1$	$1-0=1$	$1-1=0$

Example 1

Original data

10110011 ; 10101011 ; 01011010 ; 11010101

10110011	1
10101011	1
01011010	0
11010101	1
10010111	1

Row parities

Column parities

101100111 ; 101010111 ; 010110100 ; 110101011 ; 100101111

Data to be sent

Example 2

1	1	0	0	1	1	1	1	1
1	0	1	1	1	0	1	1	1
0	1	1	1	0	0	1	0	0
0	1	0	1	0	0	1	1	1
Column parities								1
Row parities								1

a. Design of row and column parities

1	1	0	0	1	1	1	1	1
1	0	1	1	1	0	1	1	1
0	1	1	1	0	0	1	0	0
0	1	0	1	0	0	1	1	1
Column parities								1
Row parities								1

b. One error affects two parities

1	1	0	0	1	1	1	1	1
1	0	1	1	1	0	1	1	1
0	1	1	1	0	0	1	0	0
0	1	0	1	0	0	1	1	1
Column parities								1
Row parities								1

c. Two errors affect two parities

1	1	0	0	1	1	1	1	1
1	0	1	1	1	0	1	1	1
0	1	1	1	0	0	1	0	0
0	1	0	1	0	0	1	1	1
Column parities								1
Row parities								1

d. Three errors affect four parities

1	1	0	0	1	1	1	1	1
1	0	1	1	1	0	1	1	1
0	1	1	1	0	0	1	0	0
0	1	0	1	0	0	1	1	1
Column parities								1
Row parities								1

e. Four errors cannot be detected

Performance of Two dimensional Parity Check

- Extra overhead is traded for better error detection capability
- Two dimensional parity check significantly improves error detection capability compared to simple parity check
- It can detect many burst errors, but not all.

CHECKSUMS

- This is a block code method where a checksum is created based on the data values in the data blocks to be transmitted using some algorithm and appended to the data.
- When the receiver gets this data, a new checksum is calculated and compared with the existing checksum.
- A non-match indicates an error.

CHECKSUM

The Sender's end

- The data is divided into k segments each of m bits
- The segments are added using ones complement arithmetic to get the sum.
- The sum is complemented to get the **checksum**
- The checksum segment is sent along with the data segments.

Example:

$$\begin{array}{r} k=4, \quad m=8 \\ 10110011 \\ 10101011 \\ \hline \curvearrowleft 01011110 \\ \quad 1 \\ \hline 01011111 \\ 01011010 \\ \hline 10111001 \\ 11010101 \\ \hline \curvearrowleft 10001110 \\ \quad 1 \\ \hline \text{Sum : } 10001111 \\ \text{Checksum } 01110000 \end{array}$$

CHECKSUM Ctd...

- The Receiver's End
- All received segments are added using ones complement arithmetic to get the sum.
- The sum is complemented
- If the result is zero, the received data is accepted; otherwise discarded.

Example: Received data

	10110011
	10101011
←	01011110
	1
	01011111
	01011010
	10111001
	11010101
←	10001110
	1
	10001111
	01110000
Sum:	11111111

Complement = 00000000
Conclusion = Accept data

Example

Sender's End

```
Frame 1:    11001100
Frame 2:    + 10101010
Partial Sum: 1 01110110
              + 1
              -----
              01110111
Frame 3:    + 11110000
Partial Sum: 1 01100111
              + 1
              -----
              01101000
Frame 4:    + 11000011
Partial Sum: 1 00101011
              + 1
              -----
Sum:        00101100
Checksum:   11010011
```

Receiver's End

```
Frame 1:    11001100
Frame 2:    + 10101010
Partial Sum: 1 01110110
              + 1
              -----
              01110111
Frame 3:    + 11110000
Partial Sum: 1 01100111
              + 1
              -----
              01101000
Frame 4:    + 11000011
Partial Sum: 1 00101011
              + 1
              -----
Sum:        00101100
Checksum:   11010011
Sum:        11111111
Complement: 00000000
Hence accept frames.
```

CHECKSUM Ctd...

Performance of Checksum

- The checksum detects all errors involving an odd number of bits
- It also detects most errors involving even number of bits.

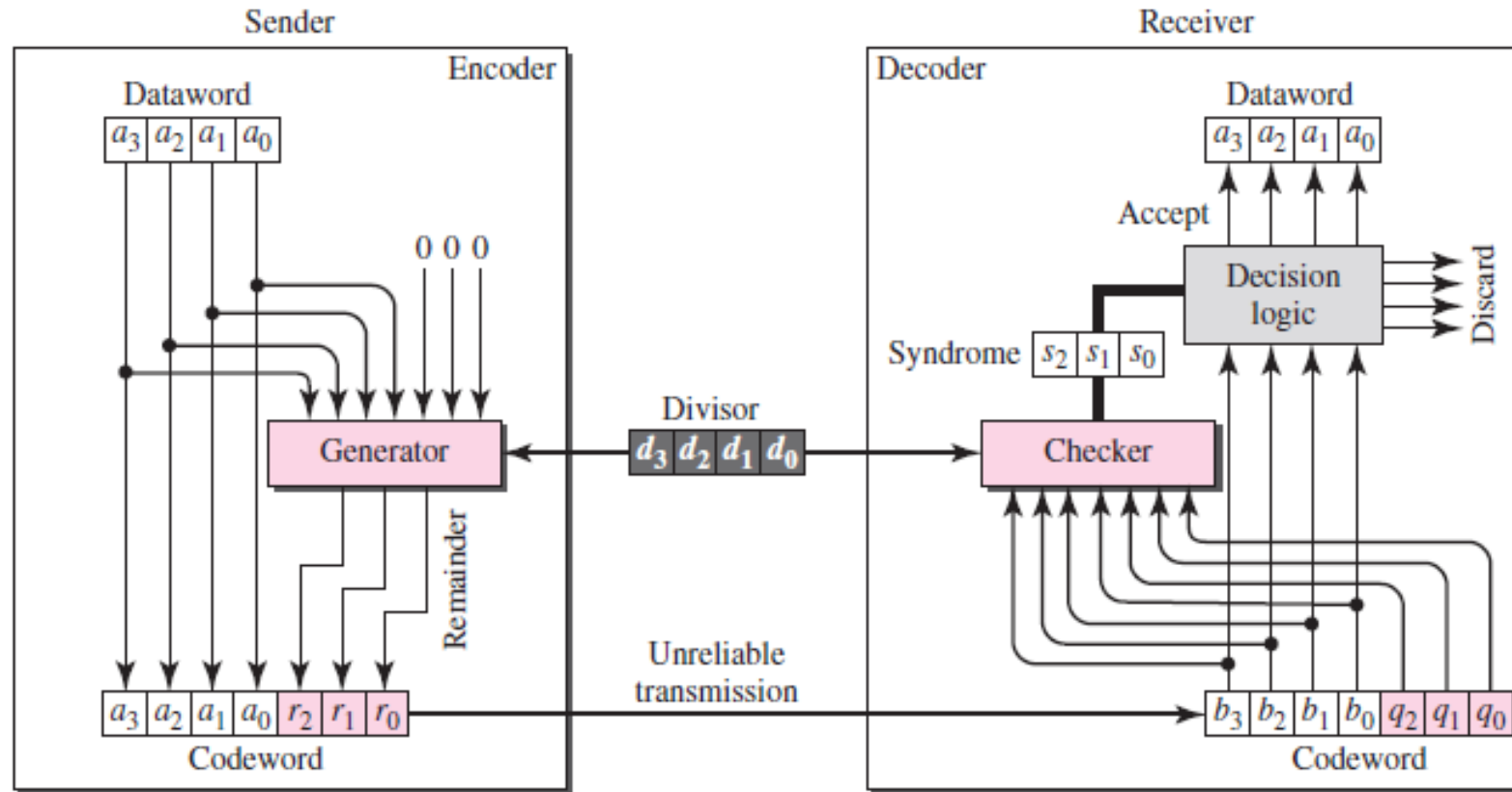
CYCLIC REDUNDANCY CHECK (CRC)

- It's among the most powerful error detecting code.
- Given a dataword of size k
- A codeword of size n
- The size of the dataword is augmented by adding $n-k$ 0s to the right hand side of the word.
- The n -bit result is fed into the generator
- The generator uses a divisor of size $n-k+1$, predefined and agreed upon
- The generator divides the augmented dataword by the divisor (Modulo-2 division).
- The quotient of the division is discarded, the remainder (r_2, r_1, r_0) is appended to the dataword to create a codeword.

CRC Ctd...

- The decoder receives the possibly corrupted codeword.
- A copy of all n bits is fed to the checker which is a replica of the generator
- The remainder produced by the checker is a Syndrome, $n-k$ bits which is fed to the decision logic analyser.
- If the Syndrome bits are all 0s, the four leftmost bits of the codeword are accepted as the dataword (interpreted as no error)
- Otherwise the four bits are discarded (error)

CRC Encoder and Decoder



CRC Ctd...

CRC Encoder

- The encoder takes the dataword and augments it with $n-k$ number of 0s or appends 0s with size of divisor minus one
- It then divides the augmented dataword by a divisor
- By using Module-2 division, if the leftmost bit of the dividend (or the part used in each step) is 0, the step cannot use the regular divisor, we need to use an all-0s divisor.
- When there are no bits left to pull down, we have a result.
- The 3-bits remainder forms the check bits (r_2, r_1, r_0) which are appended to the dataword to create a codeword.

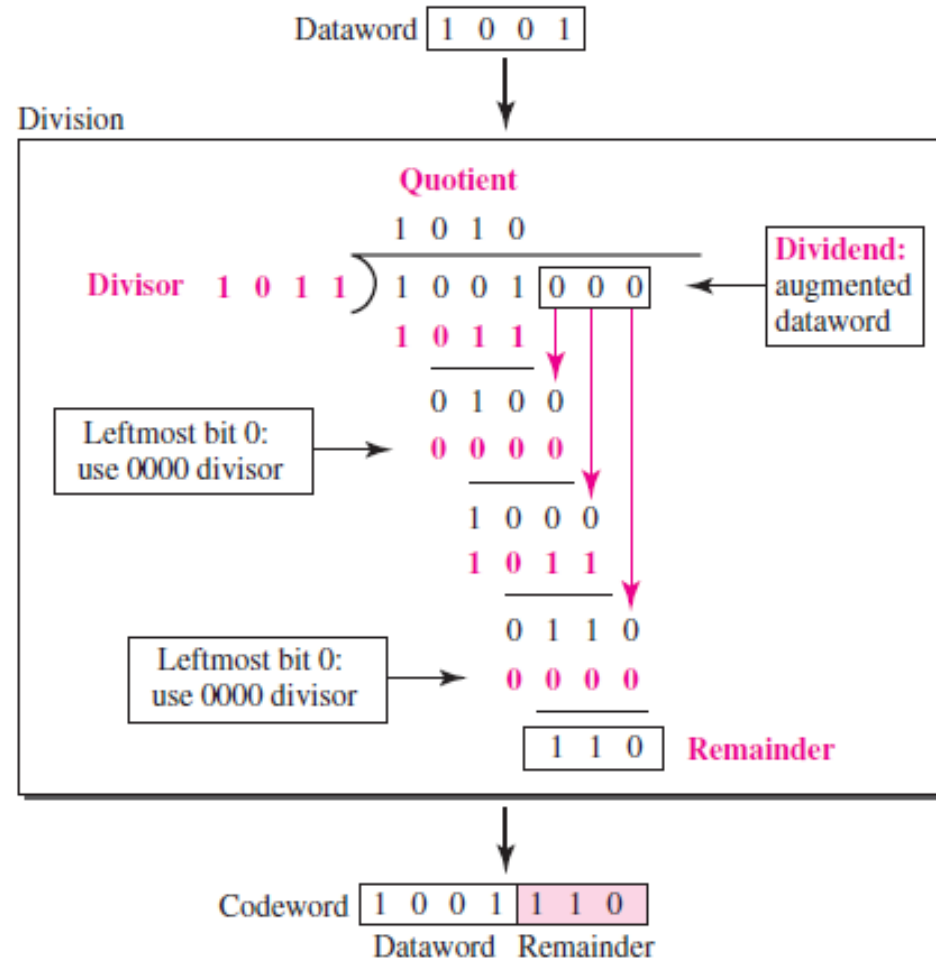
CRC Decoder

- The decoder does the same process as the encoder
- The remainder of the division is the syndrome.
- If the syndrome is all 0s, there is no error; the dataword is separated from the received codeword and accepted.
- Otherwise, everything is discarded.

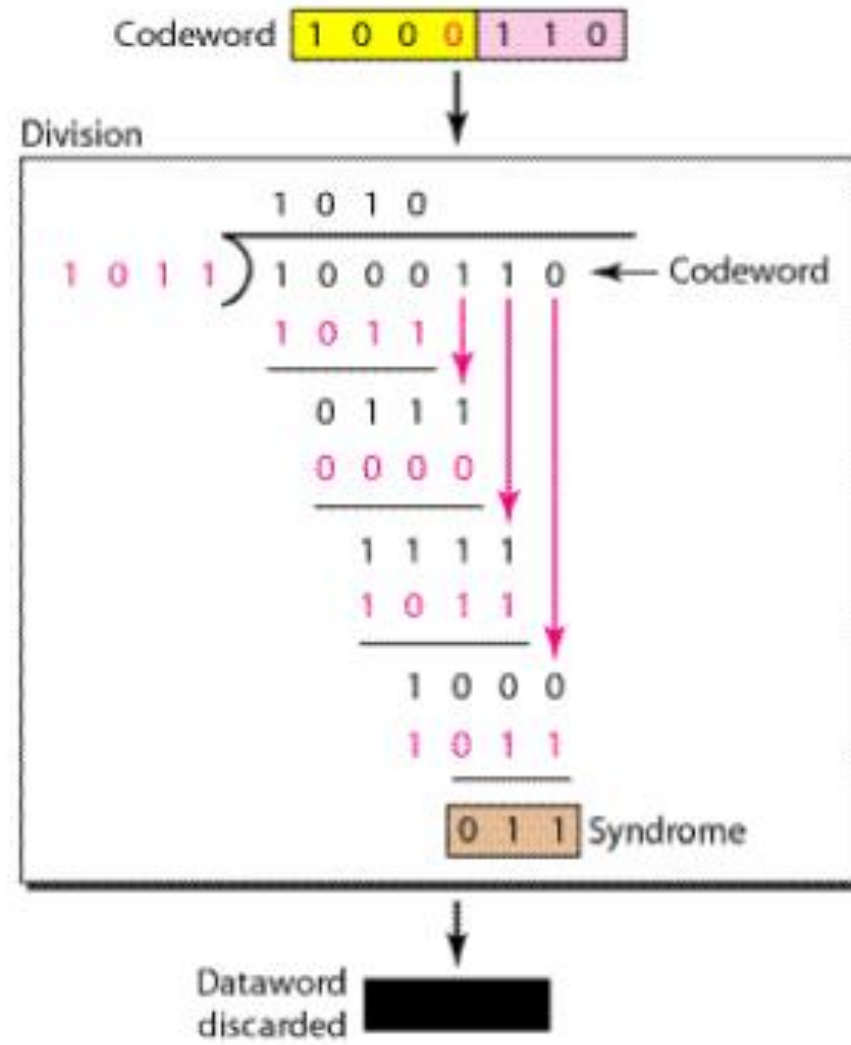
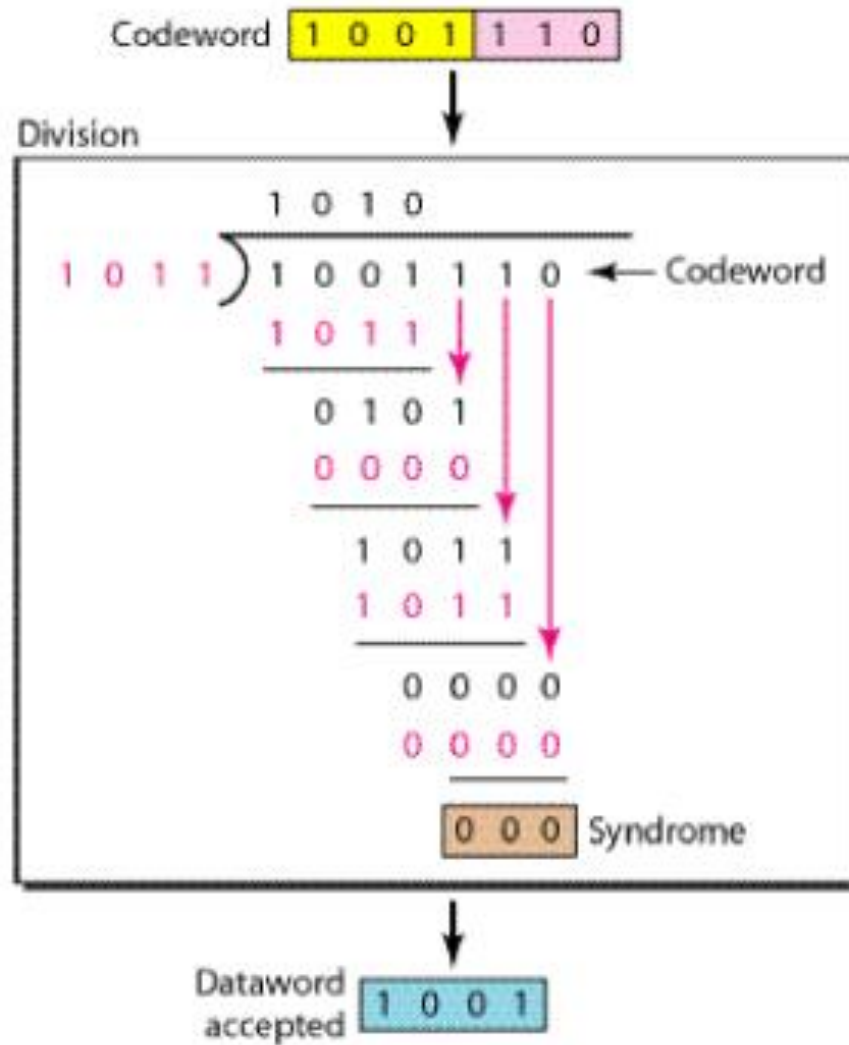
A CRC code with C(7, 4)

<i>Dataword</i>	<i>Codeword</i>	<i>Dataword</i>	<i>Codeword</i>
0000	0000000	1000	1000101
0001	0001011	1001	1001110
0010	0010110	1010	1010011
0011	0011101	1011	1011000
0100	0100111	1100	1100010
0101	0101100	1101	1101001
0110	0110001	1110	1110100
0111	0111010	1111	1111111

Example



CRC Ctd...



Example

- Given the dataword $D(x) = x^7 + x^4 + x^3 + x^1 + 1$ and the divisor $G(x) = x^3 + 1$. Show the generation of the codeword at the sender site using binary division

CRC Ctd...

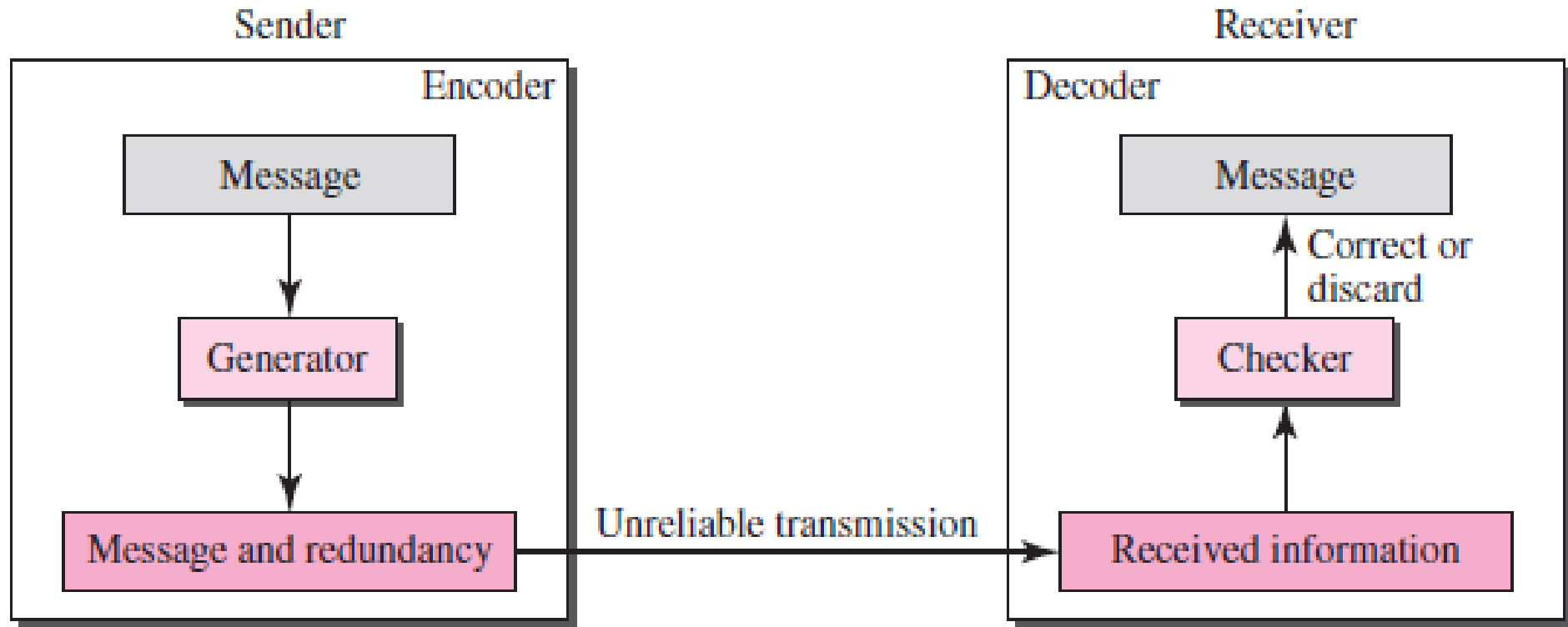
CRC Performance

- The advantage of Cyclic Redundancy Check is that it has a very good performance in detecting single-bit errors, double errors, and an odd number of errors, and burst errors.
- It can also be easily implemented in hardware and software.

ERROR CORRECTION

- While in error detection we are looking only to see if any error has occurred, In error detection we need to know the exact number of bits that are corrupted and more importantly, their location in the message.
- In error correction, the number of errors and the size of the message are the most important factors

ENCODER AND DECODER IN ERROR CORRECTION



TYPES OF ERROR CORRECTION

There are two main methods of error correction.

1. Forward error correction

Is the process in which the receiver tries to guess the message by using redundant bits. This is possible, if the number of errors is small.

2. Correction by retransmission

Is a technique in which the receiver detects the occurrence of an error and asks the sender to resend the message. Resending is repeated until a message arrives that the receiver believes is error-free (usually, not all errors can be detected).

EXAMPLE

- Consider a Codeword generated by adding three redundant bits.
- The sender consults the table (or uses an algorithm) to create the codeword **01011**
- The codeword is corrupted during transmission, and **01001** is received (error in the second bit from the right)

<i>Dataword</i>	<i>Codeword</i>
00	00000
01	01011
10	10101
11	11110

EXAMPLE Ctd...

- First, the receiver finds that the received codeword is not in the table.
- This means an error has occurred. (Detection must come before correction.)
- The receiver, assuming that there is only 1 bit corrupted, uses the following strategy to guess the correct dataword.

EXAMPLE Ctd...

1. Comparing the received codeword with the first codeword in the table (01001 versus 00000), the receiver decides that the first codeword is not the one that was sent because there are two different bits.
2. By the same reasoning, the original codeword cannot be the third or fourth one in the table.
3. The original codeword must be the second one in the table because this is the only one that differs from the received codeword by 1 bit. The receiver replaces 01001 with 01011 and consults the table to find the dataword 01.

Thank You!!!